

OPTI-CROP PROJECT DELIVERABLE

Technology Stack

1. Core Architecture

Opti-Crop is built as a self-contained, light-weight web application adhering to the Model-View-Controller (MVC) architecture.

2. Technology Breakdown

Backend & Core Logic

- **Python 3.11:** Primary programming language.
- **Flask (v3.1.1):** Micro-framework chosen for its lightweight footprint, fast routing, and clean integration with scientific Python libraries.
- **Gunicorn (v23.0.0):** Production WSGI HTTP server to execute the Flask application on hosting platforms.

Machine Learning & Data Processing

- **Scikit-Learn (v1.5.2):** Core ML library used to train and execute the Logistic Regression classifier and KMeans clustering.
- **Pandas & NumPy:** Used for loading the Kaggle crop recommendation dataset, cleaning outliers, and parsing arrays.
- **Matplotlib & Seaborn:** Used in training phase to generate analytics visualization assets (elbow graph, feature correlation heatmap, distributions).

Data Layer

- **SQLite3:** Relational database file engine. SQLite was selected because it is serverless, zero-configuration, and fully self-contained, ensuring high speed and easy local execution.
- **Pickle Binary Serialization:** Used to store the trained model (`model.pkl`), average crop statistics (`crop_ranges.pkl`), and clustered crop mappings (`cluster_insights.pkl`).

Frontend User Interface

- **HTML5 & CSS3:** Structured layout and visual styling. Built using a modern glassmorphism aesthetic with tailored dark theme variations.
- **Bootstrap (v5):** Responsive layout grid and predefined user interface components (cards, forms, alerts).